

```
myBook.set({
  title : 'Open Source',
  author : 'Pradeep Meruva',
  genre : 'Technical'
});
```

You can also set the attributes while instantiating the model itself, as shown below:

```
/* setting the attributes for the Model*/
var myBook = new Book(
{
  title : 'Open Source',
  author : 'Pradeep Meruva',
  genre : 'Technical'
});
```

The *model.get* ('attribute') method is used to retrieve the values of the model attributes as shown below:

```
var title = myBook.get("title");
var author = myBook.get("author");
var genre = myBook.get("genre");
```

## Communicating with the server

Backbone.js models also help you to communicate with the server by means of REST API calls.

Models are directly saved to the server and retrieved from it. The *id* attribute of a model plays a role of surrogate-key while interacting with the server.

To save a model call *model.save()* method, Backbone.js fires the RESTful POST request on the *urlRoot* specified in the model, with the model's attributes as payload. You need to implement the server-side mechanism to handle the corresponding RESTful request.

```
var myBook = new Book(
{
  title : 'Open Source',
  author : 'Pradeep Meruva',
  genre : 'Technical'
});

/*
save method triggers RESTful POST /books with a payload of {
  title : 'Open Source',
  author : 'Pradeep Meruva',
  genre : 'Technical'
}
*/
myBook.save();
```

If the model is already present on the server and you want to update it with the modified attributes, just set the attributes you want to modify along with the *id* attribute and call *save method*. If the *id* attribute is available, Backbone.js fires the RESTful PUT method instead of POST. This is the general RESTful convention.

```
var myBook = new Book(
{
  id : 121,
  author : 'Kumar'
});

/*
save method triggers RESTful PUT /books/121 with a payload
of {
  id : 121,
  author : 'Kumar'
}
*/
myBook.save();
```

In the same way, the *model.destroy()* method deletes the model that already exists on the server. To delete a model, you need to make sure that the model *id* attribute matches with the object *id* attribute on the server. The *Model.destroy()* method fires RESTful DELETE at the *urlRoot* provided in the model.

```
var myBook = new Book(
{
  id : 121,
});

/*
destroy method triggers RESTful DELETE /books/121 with a
payload of {
  id : 121
}
*/
myBook.destroy();
```

## View

Backbone.js's view represents the presentation of data in the form of HTML and events triggered by these HTML components. Views don't determine anything about your HTML or CSS for you, and can be used with any JavaScript templating library. The general idea is to organise your interface into logical views, backed by models, each of which can be updated independently when the model changes, without having to redraw the page. Let's create a view which displays a book and allows the user to change its attributes. In the following code, I am going to create an underscore template called *book\_template*, and then compile the template by passing the simple book model.